Unit 5 - I/O Programming & Multithreading In Java

1. INTRODUCTION TO I/O PROGRAMMING IN JAVA

1.1 WHAT IS I/O PROGRAMMING?

I/O stands for Input and Output.

I/O programming in Java is used to read data from an input source and write data to an output destination.

In simple words:

- Input: Receiving data from the user, keyboard, file or another device.
- Output: Displaying data on the screen or saving data into a file.

For example:

1. A student enters their name and marks using the keyboard.
2. The Java program receives the information.
3. The program processes the information.
4. The result is displayed on the screen or stored in a file.

This complete process is called Input/Output (I/O) programming.

EXAMPLES OF I/O OPERATIONS

1. Reading student details from the keyboard.
2. Reading employee records from a file.
3. Writing student marks into a text file.
4. Copying images from one file to another.
5. Sending and receiving data over a network.

1.2 WHAT IS A JAVA I/O STREAM?

A stream is a continuous flow of data between a source and a destination.

Java uses streams to transfer data from one location to another.

A stream acts like a connection between the program and the source or destination.

Example:

When we read a text file:

- The file acts as the input source.
- The Java program reads the data through an input stream.

When we write information into a file:

- The Java program generates the data.
- The file acts as the output destination.

TYPES OF JAVA I/O STREAMS

Java provides two main types of streams.

1. Input Stream

An input stream is used to read data from a source into a Java program.

Examples:

- Reading data from a file.
- Receiving input from a keyboard.
- Reading data from a network.

2. Output Stream

An output stream is used to write data from a Java program to a destination.

Examples:

- Writing data into a file.
- Displaying information on the screen.
- Sending information through a network.

CLASSIFICATION OF JAVA STREAMS

Java I/O streams can also be classified into two categories based on the type of data they handle.

CHARACTER STREAMS	BYTE STREAMS
Handle character data.	Handle binary data in bytes.
Mainly used for text files.	Mainly used for images, audio and other binary files.
Read and write characters.	Read and write bytes.
Examples: FileReader, FileWriter.	Examples: FileInputStream, FileOutputStream.

IMPORTANT JAVA I/O CLASSES

Java provides several classes in the java.io package.

CLASS	PURPOSE
File	Represents a file or directory.
FileInputStream	Reads bytes from a file.
FileOutputStream	Writes bytes into a file.
FileReader	Reads character data from a file.
FileWriter	Writes character data into a file.
BufferedReader	Reads text efficiently using a buffer.
BufferedWriter	Writes text efficiently using a buffer.
PrintWriter	Writes formatted text into a file.
Scanner	Reads input from keyboard or other sources.

Note: The java.io package contains classes and interfaces required to perform input and output operations in Java.

1.3 STANDARD I/O STREAMS IN JAVA

Java provides three standard I/O streams.

1. System.in

- It is the standard input stream.
- It is used to receive input from the keyboard.
- It is commonly used with the Scanner class.

Example:

```
Scanner sc = new Scanner(System.in);

int a = sc.nextInt();
```

2. System.out

- It is the standard output stream.
- It is used to display information on the screen.
- It is commonly used with print() and println().

Example:

```
System.out.println("Welcome to Java");
```

3. System.err

- It is the standard error stream.
- It is used to display error messages.
- It helps programmers identify errors during program execution.

Example:

```
System.err.println("Error occurred!");
```

2. TEXT I/O IN JAVA

2.1 WHAT IS TEXT I/O?

Text I/O stands for Text Input and Output.

It is the process of reading and writing character-based data, such as letters, numbers, words and sentences.

Java provides character stream classes to perform text I/O operations.

Text I/O is mainly used when working with text files.

Examples:

- Writing student details into a text file.
- Reading employee information from a file.
- Storing college names and addresses.
- Saving marks and attendance records.

IMPORTANT CHARACTER STREAM CLASSES

The following classes are commonly used for text I/O.

CLASS	DESCRIPTION
FileReader	Reads characters from a file.
FileWriter	Writes characters into a file.
BufferedReader	Reads text efficiently using a buffer.
BufferedWriter	Writes text efficiently using a buffer.
PrintWriter	Writes formatted text into a file.

2.2 WRITING DATA INTO A TEXT FILE USING FILEWRITER

The FileWriter class is used to write character data into a text file.

If the specified file does not exist, Java creates a new file.

If the file already exists, its previous content is overwritten by default.

SYNTAX

```
FileWriter fw = new FileWriter("filename.txt");
```

JAVA PROGRAM: WRITING DATA INTO A TEXT FILE

```
import java.io.*;
public class WriteTextFile {
    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("student.txt");
            fw.write("Name: Rahul\n");
            fw.write("Roll No: 101\n");
            fw.write("Course: BCA\n");
            fw.write("College: ABC College\n");
            fw.close();
            System.out.println("Data written successfully.");
        }
        catch(IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

EXPLANATION OF THE PROGRAM

1. import java.io.*; imports the required input/output classes.
2. FileWriter fw creates a FileWriter object.
3. new FileWriter("student.txt") creates or opens the file.
4. fw.write() writes the given information into the file.
5. \n moves the next information to a new line.
6. fw.close() closes the file after writing.
7. The catch block handles input/output errors.

Output:

Data written successfully.

Content stored in student.txt:

Name: Rahul

Roll No: 101

Course: BCA

College: ABC College

2.3 READING DATA FROM A TEXT FILE USING BUFFEREDREADER

The BufferedReader class is used to read character data efficiently from a text file.

It reads data using a temporary memory area called a buffer.

A buffer helps reduce repeated file access and improves reading performance.

The readLine() method reads one complete line at a time.

SYNTAX

```
BufferedReader br = new BufferedReader(
    new FileReader("student.txt"));
```

IMPORTANT METHODS

METHOD	DESCRIPTION
read()	Reads a single character.
readLine()	Reads one complete line.

close()	Closes the reader.

JAVA PROGRAM: READING A TEXT FILE

```
import java.io.*;


public class ReadTextFile {
   public static void main(String[] args) {


      try {
         BufferedReader br =
            new BufferedReader(
               new FileReader("student.txt"));


         String line;


         while((line = br.readLine()) != null) {
            System.out.println(line);
         }


         br.close();
      }
      catch(IOException e) {
         System.out.println("Error: " + e.getMessage());
      }
   }
}
```

EXPLANATION

1. FileReader opens the student.txt file.
2. BufferedReader reads the file efficiently.
3. A String variable named line stores each line.
4. readLine() reads one line at a time.
5. The while loop continues until the end of the file.
6. When there is no more data, readLine() returns null.
7. close() closes the file.

Output:

Name: Rahul

Roll No: 101

Course: BCA

College: ABC College

ADVANTAGES OF TEXT I/O

1. Easy to read and understand.
2. Suitable for storing normal text information.
3. Text files can be opened using common text editors.
4. Easy to modify and maintain.
5. Useful for storing student and employee records.

3. BINARY I/O IN JAVA

3.1 WHAT IS BINARY I/O?

Binary I/O is the process of reading and writing data in the form of binary information (bytes).

In Java, binary I/O is used to store and retrieve data in its original byte format.

Unlike text I/O, binary I/O does not mainly store information as readable characters.

It is useful for handling files such as:

- Images
- Audio files
- Video files
- PDF files
- Other binary data

Example:

When we save a photograph, the image is stored as binary data. Java can read and write this data using byte stream classes.

3.2 COMMON BINARY I/O CLASSES

Java provides several classes for binary input and output operations.

CLASS	TYPE	PURPOSE
FileInputStream	Input	Reads bytes from a file.
FileOutputStream	Output	Writes bytes into a file.
BufferedInputStream	Input	Reads bytes efficiently using a buffer.
BufferedOutputStream	Output	Writes bytes efficiently using a buffer.
DataInputStream	Input	Reads primitive data types.
DataOutputStream	Output	Writes primitive data types.

ObjectInputStream	Input	Reads Java objects.
ObjectOutputStream	Output	Writes Java objects.

3.3 WRITING BINARY DATA USING DATAOUTPUTSTREAM

The DataOutputStream class is used to write primitive data types into a file in binary format.

Primitive data types include:

- int
- float
- double
- boolean
- char
- long

IMPORTANT METHODS

METHOD	PURPOSE
writeInt()	Writes an integer value.
writeDouble()	Writes a double value.
writeFloat()	Writes a float value.
writeBoolean()	Writes a boolean value.
writeUTF()	Writes a string using modified UTF-8 encoding.
close()	Closes the stream.

JAVA PROGRAM: WRITING BINARY DATA

```
import java.io.*;

public class WriteBinaryFile {

    public static void main(String[] args) {

        try {

            DataOutputStream dos =

                new DataOutputStream(

                    new FileOutputStream("data.dat"));

            dos.writeInt(101);

            dos.writeDouble(85.5);

            dos.writeUTF("Rahul");

            dos.close();

            System.out.println("Binary data written successfully.");

        }

        catch(IOException e) {
```

```
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

EXPLANATION

1. DataOutputStream is created to write primitive data.
2. FileOutputStream opens or creates the data.dat file.
3. writeInt(101) writes the roll number.
4. writeDouble(85.5) writes the marks.
5. writeUTF("Rahul") writes the student's name.
6. close() closes the stream.

Output:

Binary data written successfully.

3.4 READING BINARY DATA USING DATAINPUTSTREAM

The DataInputStream class is used to read primitive data types from a binary file.

It should read the values in the same order and using the same data types in which they were written.

IMPORTANT METHODS

METHOD	PURPOSE
readInt()	Reads an integer value.
readDouble()	Reads a double value.
readFloat()	Reads a float value.
readBoolean()	Reads a boolean value.
readUTF()	Reads a string.
close()	Closes the stream.

JAVA PROGRAM: READING BINARY DATA

```
import java.io.*;

public class ReadBinaryFile {

    public static void main(String[] args) {

        try {

            DataInputStream dis =

                new DataInputStream(

                    new FileInputStream("data.dat"));

            int roll = dis.readInt();

            double marks = dis.readDouble();

            String name = dis.readUTF();

            System.out.println("Roll No: " + roll);

            System.out.println("Marks: " + marks);

            System.out.println("Name: " + name);

            dis.close();
```

```
        }
        catch(IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output:

Roll No: 101

Marks: 85.5

Name: Rahul

ADVANTAGES OF BINARY I/O

1. Suitable for storing binary files.
2. Useful for images, audio and video files.
3. Can store primitive data types directly.
4. Efficient for handling non-text data.
5. Supports object storage through object streams.

DIFFERENCE BETWEEN TEXT I/O AND BINARY I/O

TEXT I/O	BINARY I/O
Stores character-based data.	Stores data in byte format.
Mainly used for text files.	Used for images, audio and other binary files.
Human-readable.	Generally not directly human-readable.
Uses FileReader and FileWriter.	Uses FileInputStream and FileOutputStream.
Suitable for text documents.	Suitable for binary data.

4. OBJECT I/O IN JAVA

4.1 WHAT IS OBJECT I/O?

Object I/O is the process of reading and writing complete Java objects into a file.

Instead of storing individual variables separately, we can store an entire object containing multiple data members.

For example, a Student object may contain:

- Roll number
- Student name
- Marks

Using Object I/O, all these details can be stored together.

Object I/O is implemented using:

1. ObjectOutputStream
2. ObjectInputStream

These classes belong to the java.io package.

4.2 IMPORTANT CONCEPTS OF OBJECT I/O

1. ObjectOutputStream

ObjectOutputStream is used to write Java objects into a file.

It converts an object into a byte stream.

This process is called serialization.

2. ObjectInputStream

ObjectInputStream is used to read Java objects from a file.

It converts the stored byte stream back into a Java object.

This process is called deserialization.

3. Serializable Interface

Serializable is a marker interface in Java.

A marker interface does not contain any methods. It indicates that objects of the implementing class can be serialized.

To save an object using ObjectOutputStream, its class must implement Serializable.

4. Serialization

Serialization is the process of converting a Java object into a sequence of bytes so that it can be stored in a file or transferred over a network.

5. Deserialization

Deserialization is the reverse process of serialization. It converts the stored byte data back into an object.

4.3 WRITING AN OBJECT INTO A FILE USING OBJECTOUTPUTSTREAM

JAVA PROGRAM

```
import java.io.*;


class Student implements Serializable {


   int roll;

   String name;
```

```
    double marks;

    Student(int r, String n, double m) {
        roll = r;
        name = n;
        marks = m;
    }
}

public class WriteObject {
    public static void main(String[] args) {

        try {
            ObjectOutputStream oos =
                new ObjectOutputStream(
                    new FileOutputStream("student.dat"));

            Student s = new Student(101, "Rahul", 85.5);

            oos.writeObject(s);

            oos.close();

            System.out.println("Object written successfully.");
        }
        catch(IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

EXPLANATION

1. The Student class implements the Serializable interface.
2. Three data members are declared: roll, name and marks.
3. A constructor initializes the student details.
4. ObjectOutputStream is created to write objects.
5. A Student object is created.
6. writeObject(s) stores the complete object in the file.

7. close() closes the stream.

Output:

Object written successfully.

4.4 READING AN OBJECT FROM A FILE USING OBJECTINPUTSTREAM

JAVA PROGRAM

```
import java.io.*;


public class ReadObject {
   public static void main(String[] args) {


      try {
         ObjectInputStream ois =
            new ObjectInputStream(
               new FileInputStream("student.dat"));


         Student s = (Student) ois.readObject();


         System.out.println("Roll No: " + s.roll);
         System.out.println("Name: " + s.name);
         System.out.println("Marks: " + s.marks);


         ois.close();
      }
      catch(Exception e) {
         System.out.println("Error: " + e.getMessage());
      }
   }
}
```

Output:

Roll No: 101

Name: Rahul

Marks: 85.5

ADVANTAGES OF OBJECT I/O

1. Stores complete objects in a file.
2. Reduces the need to store each variable separately.
3. Useful for saving student and employee records.
4. Helps transfer objects between systems.
5. Supports serialization and deserialization.

5. RANDOM ACCESS FILES IN JAVA

5.1 WHAT IS A RANDOM ACCESS FILE?

A Random Access File is a file that allows a program to read or write data at any specific position without reading the entire file from the beginning.

Normally, sequential file access reads data one after another.

But random access allows us to directly move to a particular location in the file.

Simple Example:

Consider a student file containing 100 student records.

If we want to access the record of student number 50:

- In sequential access, we may need to read the records from the beginning.
- In random access, we can move directly to the required position, provided the record layout and position are known.

Java provides the RandomAccessFile class to perform random access operations.

5.2 IMPORTANT FEATURES OF RANDOMACCESSFILE

1. It supports both reading and writing.
2. It allows movement to any position in a file.
3. It provides methods to control the file pointer.
4. It supports different file access modes.
5. It is useful for managing records.

5.3 FILE ACCESS MODES

RandomAccessFile supports different modes.

MODE	MEANING
r	Opens the file for reading only.
rw	Opens the file for both reading and writing.
rws	Opens for reading and writing with synchronous content updates.
rwd	Opens for reading and writing with synchronous data updates.

5.4 IMPORTANT METHODS OF RANDOMACCESSFILE

METHOD	DESCRIPTION
readInt()	Reads an integer.
readDouble()	Reads a double value.
readUTF()	Reads a string.

writeInt()	Writes an integer.
writeDouble()	Writes a double value.
writeUTF()	Writes a string.
seek()	Moves the file pointer to a specified position.
getFilePointer()	Returns the current file pointer position.
length()	Returns the file size in bytes.
close()	Closes the file.

5.5 WRITING AND READING DATA USING RANDOMACCESSFILE

JAVA PROGRAM

```
import java.io.*;


public class RandomAccessDemo {
   public static void main(String[] args) {


      try {
         RandomAccessFile raf =
            new RandomAccessFile("student.dat", "rw");


         // Writing data
         raf.writeInt(101);
         raf.writeUTF("Rahul");
         raf.writeDouble(85.5);


         // Moving to the beginning
         raf.seek(0);


         // Reading data
         int roll = raf.readInt();
         String name = raf.readUTF();
         double marks = raf.readDouble();


         System.out.println("Roll No: " + roll);
         System.out.println("Name: " + name);
         System.out.println("Marks: " + marks);


         raf.close();
```

```
        }
        catch(IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

EXPLANATION

1. A RandomAccessFile object is created.
2. The file is opened using the rw mode.
3. writeInt() writes the roll number.
4. writeUTF() writes the student's name.
5. writeDouble() writes the marks.
6. seek(0) moves the file pointer to the beginning.
7. The read methods retrieve the stored information.
8. The retrieved information is displayed.
9. Finally, the file is closed.

Output:

Roll No: 101

Name: Rahul

Marks: 85.5

ADVANTAGES OF RANDOM ACCESS FILES

1. Allows direct access to data.
2. Supports both reading and writing.
3. Saves time when accessing specific records.
4. Useful for database-like record management.
5. Provides control over the file pointer.

DIFFERENCE BETWEEN SEQUENTIAL ACCESS AND RANDOM ACCESS

SEQUENTIAL ACCESS	RANDOM ACCESS
Data is accessed in order.	Data can be accessed at any position.
Reads data one after another.	Can directly move to a required position.
Suitable for simple text processing.	Suitable for record-based applications.
Usually uses streams and readers.	Uses RandomAccessFile.

6. MULTITHREADING IN JAVA

6.1 WHAT IS MULTITHREADING?

Multithreading is a feature of Java that allows multiple threads to execute concurrently within a single program.

A thread is a small unit of execution within a process.

In simple words, multithreading allows a program to perform multiple tasks at the same time.

Each thread performs a separate task, but all threads share the resources of the same process.

Real-World Example:

Consider a music application:

- One thread plays the music.
- Another thread downloads songs.
- Another thread handles user interactions.

All these tasks can work concurrently without stopping the entire application.

6.2 WHAT IS A THREAD?

A thread is an independent path of execution within a program.

A Java program can contain multiple threads.

The main thread is automatically created when a Java program starts.

Additional threads can be created by the programmer.

DIFFERENCE BETWEEN PROCESS AND THREAD

PROCESS	THREAD
A process is a running program.	A thread is a unit of execution within a process.
Has its own memory space.	Shares process memory with other threads.
Generally requires more resources.	Generally requires fewer resources.
A process can contain multiple threads.	A thread belongs to a process.

6.3 WHY IS MULTITHREADING REQUIRED?

Multithreading provides several advantages.

1. Improved Performance

Multiple threads can perform different tasks concurrently, improving application efficiency.

2. Better Responsiveness

An application can continue responding to user actions even when another task is running.

3. Resource Utilization

Threads can make better use of available CPU resources.

4. Concurrent Task Execution

Multiple tasks such as downloading, printing and processing data can execute concurrently.

5. Efficient Application Development

It helps develop applications that perform multiple operations without unnecessarily blocking each other.

6.4 WAYS TO CREATE THREADS IN JAVA

Java provides two commonly used ways to create threads.

1. By extending the Thread class.
2. By implementing the Runnable interface.

METHOD 1: CREATING A THREAD BY EXTENDING THE THREAD CLASS

In this method, we create a class that extends the Thread class and override its run() method.

The run() method contains the code that will be executed by the new thread.

JAVA PROGRAM

```
class MyThread extends Thread {


   public void run() {

      System.out.println("Child thread is running");

   }

}


public class Main {

   public static void main(String[] args) {


      MyThread t = new MyThread();


      t.start();


      System.out.println("Main thread is running");

   }

}
```

EXPLANATION

1. MyThread extends the Thread class.
2. The run() method is overridden.
3. A thread object is created.
4. The start() method starts a new thread.
5. The JVM schedules the child thread to execute its run() method.
6. The main thread also continues execution.

Possible Output:

Main thread is running

Child thread is running

Important Note: The output order may change because both threads execute concurrently.

METHOD 2: CREATING A THREAD BY IMPLEMENTING RUNNABLE INTERFACE

The Runnable interface is used to define a task that can be executed by a thread.

It contains one abstract method called run().

JAVA PROGRAM

```
class MyTask implements Runnable {
    public void run() {
        System.out.println("Child thread is running");
    }
}
public class TestRunnable {
    public static void main(String[] args) {
        MyTask task = new MyTask();
        Thread t = new Thread(task);
        t.start();
        System.out.println("Main thread is running");
    }
}
```

EXPLANATION

1. MyTask implements the Runnable interface.
2. The run() method is implemented.
3. A MyTask object is created.
4. The object is passed to the Thread constructor.
5. The start() method starts the new thread.
6. The run method executes in the new thread.
7.

DIFFERENCE BETWEEN THREAD CLASS AND RUNNABLE INTERFACE

THREAD CLASS	RUNNABLE INTERFACE

A class extends Thread.	A class implements Runnable.
The run() method is overridden.	The run() method is implemented.
The class cannot extend another class because Java supports single inheritance.	The class can extend another class.
Task and thread are combined in one class.	Task and thread are separated.
Less flexible for reusable tasks.	More flexible and commonly preferred for defining tasks.

7. THREAD LIFE CYCLE IN JAVA

7.1 WHAT IS THREAD LIFE CYCLE?

The thread life cycle represents the different states through which a thread passes during its execution.

A thread does not remain in the same state throughout its lifetime.

It moves from one state to another depending on its execution, scheduling and synchronization.

Java provides the Thread.State enumeration to represent thread states.

7.2 DIFFERENT STATES OF A THREAD

Java defines six thread states.

1. NEW STATE

A thread is in the NEW state when a thread object is created but the start() method has not been called.

Example:

Thread t = new Thread();

At this stage, the thread has not started execution.

2. RUNNABLE STATE

A thread enters the RUNNABLE state after the start() method is called.

It is ready to run or may be running under the JVM's thread scheduling.

Example:

t.start();

3. BLOCKED STATE

A thread enters the BLOCKED state when it is waiting to acquire a lock that is currently held by another thread.

Example:

Two threads are trying to access the same synchronized method. One thread holds the lock, while the other waits.

4. WAITING STATE

A thread enters the WAITING state when it waits indefinitely for another thread to perform a particular action.

Some methods that can cause this state are:

- wait()
- join()

5. TIMED_WAITING STATE

A thread enters the TIMED_WAITING state when it waits for a specified period.

Some methods include:

- sleep()
- wait(time)
- join(time)

6. TERMINATED STATE

A thread enters the TERMINATED state when its execution is completed or it stops due to an uncaught exception.

Once a thread is terminated, it cannot be restarted.

7.3 THREAD LIFE CYCLE PROGRAM

```
class MyThread extends Thread {


   public void run() {


      for(int i = 1; i <= 5; i++) {


         System.out.println("Child Thread: " + i);


         try {
            Thread.sleep(1000);
         }
         catch(InterruptedException e) {
            System.out.println("Thread interrupted");
         }
      }
   }
}
```

```
public class TestThread {
   public static void main(String[] args) {

      MyThread t = new MyThread();

      System.out.println("State after creation: " + t.getState());

      t.start();

      System.out.println("State after start: " + t.getState());

      try {
         t.join();
      }
      catch(InterruptedException e) {
         System.out.println(e);
      }

      System.out.println("State after completion: " + t.getState());
   }
}
```

IMPORTANT THREAD METHODS

METHOD	DESCRIPTION
start()	Starts a new thread.
run()	Contains the task performed by the thread.
sleep()	Makes the current thread pause for a specified time.
join()	Makes the calling thread wait until another thread finishes.
getState()	Returns the current thread state.
getName()	Returns the thread name.

setName()	Changes the thread name.
isAlive()	Checks whether a thread is still alive.
currentThread()	Returns the currently executing thread.
interrupt()	Interrupts a thread's waiting or sleeping operation, or sets its interrupt status.

Important: Calling run() directly does not start a new thread. The start() method is used to begin a new thread.

8. METHODS IN JAVA

8.1 WHAT IS A METHOD?

A method in Java is a block of code that performs a specific task.

Methods help divide a large program into smaller and manageable parts.

A method can be called whenever its functionality is required.

For example, in a student management system, we can create separate methods for:

- Reading student details.
- Calculating total marks.
- Calculating percentage.
- Displaying student results.

Instead of writing the same code repeatedly, we can define a method once and call it multiple times.

8.2 ADVANTAGES OF METHODS

1. Code Reusability: A method can be called multiple times.
2. Reduces Code Duplication: Avoids writing the same code repeatedly.
3. Improves Readability: Makes programs easier to understand.
4. Easy Maintenance: Changes can be made in one method.
5. Modularity: Divides large programs into smaller sections.
6. Easy Testing: Individual methods can be tested separately.

8.3 SYNTAX OF A METHOD

```
returnType methodName(parameters) {


    // Statements


    return value;
}
```

EXPLANATION

- returnType: Specifies the type of value returned by the method.
- methodName: Specifies the name of the method.
- parameters: Values received by the method.
- return: Sends the result back to the calling method.

Example:

```
int add(int a, int b) {
   return a + b;
}
```

Here:

- int is the return type.
- add is the method name.
- a and b are parameters.
- return a + b returns the sum.

8.4 TYPES OF METHODS IN JAVA

Based on parameters and return values, methods can be divided into four categories.

1. VOID METHOD (WITHOUT PARAMETERS)

A void method does not return any value and does not accept parameters.

Example:

```
void display() {
   System.out.println("Welcome to Java");
}
```

2. RETURN TYPE METHOD (WITHOUT PARAMETERS)

A return type method returns a value but does not accept parameters.

Example:

```
int getNumber() {
   return 100;
}
```

3. PARAMETERIZED METHOD

A parameterized method accepts values from the calling method.

Example:

```
void display(int n) {
```

```
    System.out.println("Number: " + n);
}
```

4. PARAMETERIZED METHOD WITH RETURN TYPE

This method accepts parameters and also returns a value.

Example:

```
int add(int a, int b) {
    return a + b;
}
```

8.5 EXAMPLE PROGRAMS

A. VOID METHOD

```
class Demo1 {

    void showMessage() {
        System.out.println("Hello from Method!");
    }

    public static void main(String[] args) {

        Demo1 d = new Demo1();

        d.showMessage();
    }
}
```

Output:

Hello from Method!

B. PARAMETERIZED METHOD

```
class Demo2 {

    void printNumber(int n) {
        System.out.println("Number: " + n);
    }
```

```
    public static void main(String[] args) {

        Demo2 d = new Demo2();

        d.printNumber(5);
    }
}
```

Output:

Number: 5

C. METHOD WITH RETURN TYPE

```
class Demo3 {

    int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {

        Demo3 d = new Demo3();

        int result = d.add(10, 20);

        System.out.println("Sum: " + result);
    }
}
```

Output:

Sum: 30

IMPORTANT POINTS ABOUT METHODS

1. A method must be declared inside a class.
2. A method is executed when it is called.
3. A method can be called multiple times.
4. A method may accept parameters.
5. A method may return a value.
6. The main() method is the entry point of a standard Java application.

9. RUNNABLE INTERFACE IN JAVA

9.1 WHAT IS RUNNABLE INTERFACE?

Runnable is a functional interface in Java used to define a task that can be executed by a thread.

It belongs to the java.lang package.

It contains one abstract method:

public void run();

The run() method contains the instructions that should be executed by the thread.

The Runnable interface provides a flexible way to create threads because a class can implement Runnable and still extend another class.

9.2 DECLARATION OF RUNNABLE INTERFACE

```
@FunctionalInterface
public interface Runnable {

   public void run();

}
```

9.3 IMPLEMENTING RUNNABLE INTERFACE

To create a thread using Runnable:

1. Create a class that implements Runnable.
2. Implement the run() method.
3. Create an object of the class.
4. Pass the object to the Thread constructor.
5. Call the start() method.

9.4 JAVA PROGRAM USING RUNNABLE INTERFACE

```
class MyTask implements Runnable {

   public void run() {

      for(int i = 1; i <= 5; i++) {

         System.out.println("Child Thread: " + i);

         try {
            Thread.sleep(500);
         }
```

```
            catch(InterruptedException e) {
                System.out.println("Thread interrupted");
            }
        }
    }
}

public class TestRunnable {

    public static void main(String[] args) {

        MyTask task = new MyTask();

        Thread t = new Thread(task);

        t.start();

        System.out.println("Main thread running");
    }
}
```

Possible Output:

Main thread running

Child Thread: 1

Child Thread: 2

Child Thread: 3

Child Thread: 4

Child Thread: 5

The exact output order may vary because thread execution is controlled by the JVM scheduler.

9.5 ADVANTAGES OF RUNNABLE INTERFACE

1. It provides a flexible way to create threads.
2. It separates the task from the thread object.
3. A class can implement multiple interfaces.
4. It supports code reusability.
5. It is suitable for applications involving multiple threads.
6. It works well with Java's Executor framework.

9.6 DIFFERENCE BETWEEN THREAD AND RUNNABLE

THREAD	RUNNABLE
Thread is a class.	Runnable is an interface.
A class extends Thread.	A class implements Runnable.
Contains thread-related functionality.	Defines the task to be executed.
Java single inheritance applies.	Allows a class to extend another class.
Task and thread are combined.	Task and thread are separated.

10. THREAD SYNCHRONIZATION IN JAVA

10.1 WHAT IS THREAD SYNCHRONIZATION?

Thread synchronization is a mechanism used to control the access of multiple threads to a shared resource.

When two or more threads access the same data simultaneously, incorrect results may occur.

Synchronization ensures that only one thread at a time executes a particular synchronized section using the same lock.

Simple Example:

Consider a bank account with a balance of ₹10,000.

Two threads try to withdraw ₹7,000 at the same time.

Without proper synchronization, both threads may read the same initial balance and perform incorrect updates.

Synchronization helps prevent such problems by controlling access to shared data.

10.2 WHY IS THREAD SYNCHRONIZATION REQUIRED?

1. Prevents Data Inconsistency: Ensures shared data is updated correctly.
2. Avoids Race Conditions: Prevents unexpected results caused by simultaneous access.
3. Maintains Data Integrity: Protects important information.
4. Controls Resource Access: Allows only one thread at a time to enter a synchronized section using the same lock.
5. Improves Reliability: Helps applications produce consistent results.

10.3 WAYS TO ACHIEVE THREAD SYNCHRONIZATION

Java provides different ways to synchronize threads.

1. SYNCHRONIZED METHOD

A synchronized method allows only one thread at a time to execute that method on the same object.

Syntax:

```
synchronized void methodName() {
   // Statements
}
```

2. SYNCHRONIZED BLOCK

A synchronized block is used to synchronize only a specific portion of code instead of the entire method.

Syntax:

```
synchronized(object) {
   // Statements
}
```

3. STATIC SYNCHRONIZATION

A static synchronized method uses the class-level lock.

It is useful when multiple threads access shared static data.

Syntax:

```
static synchronized void methodName() {
   // Statements
}
```

10.4 JAVA PROGRAM USING SYNCHRONIZED METHOD

```
class Counter {

   int count = 0;

   synchronized void increment() {

      count++;

      System.out.println("Count: " + count);
   }
}

class MyThread extends Thread {

   Counter c;

   MyThread(Counter c) {
      this.c = c;
```

```
    }

    public void run() {

        for(int i = 1; i <= 5; i++) {
            c.increment();
        }
    }
}

public class TestSync {

    public static void main(String[] args) {

        Counter c = new Counter();

        MyThread t1 = new MyThread(c);
        MyThread t2 = new MyThread(c);

        t1.start();
        t2.start();
    }
}
```

EXPLANATION

1. Counter is a class that contains a shared variable named count.
2. The increment() method is declared as synchronized.
3. Two threads are created using the same Counter object.
4. Both threads call the increment() method.
5. Because the method is synchronized, only one thread at a time can execute it on that object.
6. This prevents simultaneous updates from causing a race condition.

Possible Output:

Count: 1

Count: 2

Count: 3

Count: 4

Count: 5

Count: 6

Count: 7

Count: 8

Count: 9

Count: 10

The order of thread execution may vary, but the final count will be 10.

10.5 ADVANTAGES OF THREAD SYNCHRONIZATION

1. Prevents race conditions.
2. Maintains data consistency.
3. Protects shared resources.
4. Avoids incorrect data updates.
5. Helps develop reliable multithreaded applications.

DIFFERENCE BETWEEN SYNCHRONIZED METHOD AND SYNCHRONIZED BLOCK

SYNCHRONIZED METHOD	SYNCHRONIZED BLOCK
Synchronizes the entire method.	Synchronizes only a selected block.
Easy to implement.	Provides more control.
May reduce concurrency if the method is large.	Can improve concurrency by reducing the locked code section.
Uses the method's associated object or class lock.	Can explicitly specify the lock object.

11. EXCEPTION HANDLING IN JAVA

11.1 WHAT IS EXCEPTION HANDLING?

Exception handling is a mechanism in Java used to handle runtime errors so that the normal flow of a program can be maintained.

An exception is an unexpected event that occurs during program execution.

When an exception occurs, the normal execution of the program may be interrupted.

Java provides exception-handling keywords to handle such situations.

Real-World Example:

Suppose a program divides one number by another number.

If the second number is zero, an ArithmeticException occurs.

Instead of allowing the program to terminate unexpectedly, we can handle the exception and display a suitable message.

11.2 TYPES OF ERRORS

Errors in programming can generally be classified into three categories.

1. Compile-Time Errors

These errors occur when the compiler finds mistakes in the program.

Examples:

- Missing semicolon.
- Incorrect variable declaration.
- Missing brackets.
- Invalid syntax.

2. Runtime Errors

These errors occur while the program is executing.

Examples:

- Dividing by zero.
- Accessing an invalid array index.
- Opening a file that does not exist.

3. Logical Errors

These errors occur when the program executes successfully but produces incorrect results because of incorrect logic.

Example:

```
int a = 10;

int b = 20;


int sum = a - b;
```

The program compiles and executes, but the programmer has used subtraction instead of addition.

11.3 TYPES OF EXCEPTIONS IN JAVA

Exceptions are broadly classified into two types.

1. CHECKED EXCEPTIONS

Checked exceptions are checked by the compiler during compilation.

The programmer must handle them using try-catch or declare them using the throws keyword where required.

Examples:

- IOException
- FileNotFoundException
- SQLException
- ClassNotFoundException

2. UNCHECKED EXCEPTIONS

Unchecked exceptions occur during program execution.

They are not required to be handled or declared by the compiler.

Examples:

- ArithmeticException
- NullPointerException
- ArrayIndexOutOfBoundsException
- NumberFormatException

11.4 KEYWORDS USED IN EXCEPTION HANDLING

Java provides five important exception-handling keywords.

1. TRY

The try block contains the code that may generate an exception.

Example:

```
try {
   int result = 10 / 0;
}
```

2. CATCH

The catch block is used to handle an exception generated inside the try block.

Example:

```
catch(ArithmeticException e) {
   System.out.println("Cannot divide by zero");
}
```

3. FINALLY

The finally block contains code that executes after the try-catch processing, whether an exception occurs or not, subject to JVM termination and other exceptional circumstances.

It is commonly used for cleanup operations.

Example:

```
finally {
    System.out.println("Program completed");
}
```

4. THROW

The throw keyword is used to explicitly throw an exception.

Example:

```
throw new ArithmeticException("Invalid operation");
```

5. THROWS

The throws keyword is used in a method declaration to indicate exceptions that may be passed to the calling method.

Example:

```
void readFile() throws IOException {
    // File handling code
}
```

11.5 SYNTAX OF EXCEPTION HANDLING

```
try {

    // Code that may generate an exception

}
catch(ExceptionType e) {

    // Exception handling code

}
finally {

    // Cleanup code

}
```

11.6 FLOW OF EXCEPTION HANDLING

The execution of exception handling follows these steps:

1. The program starts execution.
2. The statements inside the try block are executed.
3. If no exception occurs, the catch block is skipped.
4. If an exception occurs, the matching catch block executes.
5. The finally block executes after the try-catch processing, if present.
6. Program execution continues or terminates depending on the remaining code.

11.7 JAVA PROGRAM USING TRY-CATCH-FINALLY

```
public class ExceptionDemo {


   public static void main(String[] args) {


      int a = 10;

      int b = 0;


      try {


         int c = a / b;


         System.out.println("Result: " + c);

      }

      catch(ArithmeticException e) {


         System.out.println("Exception: " + e);

      }

      finally {


         System.out.println("Finally block executed");

      }

   }

}
```

Output:

Exception: java.lang.ArithmeticException: / by zero

Finally block executed

EXPLANATION

1. Two integer variables are declared.
2. The try block attempts to divide 10 by 0.
3. Division by zero generates ArithmeticException.
4. The catch block handles the exception.
5. The finally block executes.
6. The program avoids an unhandled exception.

11.8 COMMON EXCEPTIONS IN JAVA

EXCEPTION	CAUSE	EXAMPLE
ArithmeticException	Invalid arithmetic operation.	10 / 0
NullPointerException	Accessing a member using a null reference.	obj.display() when obj is null
ArrayIndexOutOfBoundsException	Accessing an invalid array index.	arr[10] when array size is 5
NumberFormatException	Converting invalid text into a number.	Integer.parseInt("ABC")
IOException	Input/output operation failure.	File reading error
FileNotFoundException	Required file cannot be found.	Opening a missing file

11.9 ADVANTAGES OF EXCEPTION HANDLING

1. Prevents unexpected program termination in many error situations.
2. Maintains the normal flow of program execution where recovery is possible.
3. Separates error-handling code from normal code.
4. Improves program reliability.
5. Helps identify and debug errors.
6. Provides meaningful error messages.

12. COLLECTIONS IN JAVA

12.1 WHAT ARE COLLECTIONS IN JAVA?

Collections in Java are a framework used to store, organize, manage and manipulate groups of objects.

A collection allows us to store multiple objects in a single data structure.

For example, suppose a college has 100 students. Instead of creating 100 separate variables, we can use a collection to store all student names together.

Java Collections Framework is available in the java.util package.

It provides different interfaces and classes to manage data efficiently.

Examples:

- Storing student names.
- Maintaining employee records.
- Managing product lists.

- Storing unique roll numbers.
- Maintaining a list of examination results.

12.2 WHY ARE COLLECTIONS REQUIRED?

Collections are useful because:

1. They allow us to store multiple objects.
2. They provide dynamic storage.
3. They make data management easier.
4. They provide methods for sorting and searching.
5. They help remove duplicate elements where required.
6. They reduce programming complexity.
7. They support different ways of organizing data.

12.3 COLLECTION FRAMEWORK HIERARCHY

The Java Collections Framework contains different interfaces and implementation classes.

The main interfaces are:

1. List
2. Set
3. Queue
4. Map

A. LIST INTERFACE

A List is an ordered collection that allows duplicate elements.

The elements are maintained in a particular sequence and can be accessed using their index positions.

Important List Classes:

- ArrayList
- LinkedList
- Vector
- Stack

Example:

```
import java.util.*;


public class ListExample {


   public static void main(String[] args) {


      ArrayList<String> names = new ArrayList<>();


      names.add("Rahul");

      names.add("Priya");
```

```
        names.add("Amit");
        names.add("Rahul");

        System.out.println(names);
    }
}
```

Output:

[Rahul, Priya, Amit, Rahul]

Here, Rahul appears twice because List allows duplicate elements.

B. SET INTERFACE

A Set is a collection that does not allow duplicate elements.

It is useful when we need to store unique values.

Important Set Classes:

- HashSet
- LinkedHashSet
- TreeSet

Example:

```
import java.util.*;

public class SetExample {

    public static void main(String[] args) {

        HashSet<String> names = new HashSet<>();

        names.add("Rahul");
        names.add("Priya");
        names.add("Amit");
        names.add("Rahul");

        System.out.println(names);
    }
}
```

Possible Output:

[Rahul, Priya, Amit]

The duplicate value Rahul is stored only once.

Note: HashSet does not guarantee insertion order.

C. QUEUE INTERFACE

A Queue is a collection generally used to store elements in First In First Out (FIFO) order.

FIFO means the element inserted first is normally removed first.

Real-World Example:

Consider students standing in a college fee payment queue. The student who comes first generally gets served first.

Important Queue Classes:

- PriorityQueue
- LinkedList
- ArrayDeque

Example:

```
import java.util.*;


public class QueueExample {


   public static void main(String[] args) {


      Queue<String> q = new LinkedList<>();


      q.add("Rahul");
      q.add("Priya");
      q.add("Amit");


      System.out.println(q);


      q.remove();


      System.out.println("After removal: " + q);
   }
}
```

Output:

[Rahul, Priya, Amit]

After removal: [Priya, Amit]

D. MAP INTERFACE

A Map is used to store data in the form of key-value pairs.

Each key is unique and is associated with a value.

Real-World Example:

In a college student management system:

Key	Value
101	Rahul
102	Priya
103	Amit

Here, roll numbers are keys and student names are values.

Important Map Classes:

- HashMap
- LinkedHashMap
- TreeMap
- Hashtable

Example:

```
import java.util.*;


public class MapExample {


   public static void main(String[] args) {


      HashMap<Integer, String> students = new HashMap<>();


      students.put(101, "Rahul");

      students.put(102, "Priya");

      students.put(103, "Amit");


      System.out.println(students);


      System.out.println("Student: " + students.get(101));

   }

}
```

Possible Output:

{101=Rahul, 102=Priya, 103=Amit}

Student: Rahul

12.4 IMPORTANT COLLECTION INTERFACES AND CLASSES

1. COLLECTION INTERFACE

Collection is the root interface of the main collection hierarchy.

It provides common operations such as adding, removing and checking elements.

Important methods:

- add()
- remove()
- size()
- contains()
- clear()
- isEmpty()

2. LIST INTERFACE

List stores ordered elements and allows duplicates.

Important classes:

- ArrayList
- LinkedList
- Vector
- Stack

3. ARRAYLIST

ArrayList is a resizable array implementation of the List interface.

It allows duplicate elements and maintains insertion order.

Features:

1. Dynamic size.
2. Allows duplicate values.
3. Maintains insertion order.
4. Provides fast access using indexes.
5. Useful when searching and retrieving elements frequently.

Example:

```
import java.util.*;


public class ArrayListDemo {


   public static void main(String[] args) {


      ArrayList<String> list = new ArrayList<>();


      list.add("Java");
      list.add("Python");
      list.add("C++");


      System.out.println("List: " + list);


      list.remove("Python");


      System.out.println("After removal: " + list);
   }
}
```

Output:

List: [Java, Python, C++]

After removal: [Java, C++]

4. LINKEDLIST

LinkedList is a collection class that stores elements as linked nodes.

Each node contains data and links to other nodes.

It implements both List and Deque interfaces.

Features:

1. Maintains insertion order.
2. Allows duplicate elements.
3. Supports insertion and deletion at different positions.
4. Can be used as a list or queue.
5. Does not provide direct indexed access as efficiently as ArrayList.

Example:

```
import java.util.*;
```

```
public class LinkedListDemo {

    public static void main(String[] args) {

        LinkedList<String> list = new LinkedList<>();

        list.add("Java");
        list.add("Python");
        list.add("C");

        System.out.println(list);

        list.addFirst("HTML");
        list.addLast("PHP");

        System.out.println("Updated List: " + list);
    }
}
```

Output:

[Java, Python, C]

Updated List: [HTML, Java, Python, C, PHP]

5. VECTOR

Vector is a dynamic array implementation that is synchronized.

It allows duplicate elements and maintains insertion order.

Features:

1. Automatically increases its capacity when required.
2. Allows duplicate elements.
3. Maintains insertion order.
4. Provides synchronized methods.
5. Supports dynamic storage.

6. STACK

Stack is a data structure that follows the LIFO principle.

LIFO stands for Last In First Out.

The last element inserted is the first element removed.

Real-World Example:

A stack of plates:

- The last plate placed on top is removed first.

Important Stack Methods:

METHOD	DESCRIPTION
push()	Adds an element to the stack.
pop()	Removes the top element.
peek()	Returns the top element without removing it.
empty()	Checks whether the stack is empty.
search()	Searches for an element.

Example:

```
import java.util.*;


public class StackDemo {


   public static void main(String[] args) {


      Stack<Integer> s = new Stack<>();


      s.push(10);
      s.push(20);
      s.push(30);


      System.out.println("Stack: " + s);


      System.out.println("Top element: " + s.peek());


      s.pop();
```

```
        System.out.println("After pop: " + s);
    }
}
```

Output:

Stack: [10, 20, 30]

Top element: 30

After pop: [10, 20]

12.5 DIFFERENCE BETWEEN ARRAYLIST, LINKEDLIST, VECTOR AND STACK

ARRAYLIST	LINKEDLIST	VECTOR	STACK
Dynamic array.	Linked-node structure.	Synchronized dynamic array.	LIFO data structure.
Allows duplicates.	Allows duplicates.	Allows duplicates.	Allows duplicates.
Maintains insertion order.	Maintains insertion order.	Maintains insertion order.	Follows LIFO order.
Not synchronized by default.	Not synchronized by default.	Synchronized.	Inherits synchronized methods from Vector.
Suitable for frequent indexed access.	Suitable for insertion and deletion operations.	Suitable when synchronized dynamic-array operations are needed.	Suitable for stack operations.

12.6 IMPORTANT SET CLASSES

1. HASHSET

HashSet stores unique elements and does not guarantee insertion order.

Example:

```
import java.util.*;


public class HashSetDemo {


    public static void main(String[] args) {


        HashSet<Integer> set = new HashSet<>();


        set.add(10);

        set.add(20);

        set.add(30);

        set.add(10);
```

```
        System.out.println(set);
    }
}
```

Output: The set contains 10, 20 and 30, in no guaranteed order.

2. LINKEDHASHSET

LinkedHashSet stores unique elements and maintains insertion order.

Example:

```
import java.util.*;

public class LinkedHashSetDemo {

    public static void main(String[] args) {

        LinkedHashSet<String> set = new LinkedHashSet<>();

        set.add("Java");
        set.add("Python");
        set.add("C++");

        System.out.println(set);
    }
}
```

Output:

[Java, Python, C++]

3. TREESET

TreeSet stores unique elements in sorted order.

Example:

```
import java.util.*;
```

```
public class TreeSetDemo {

   public static void main(String[] args) {

      TreeSet<Integer> set = new TreeSet<>();

      set.add(30);
      set.add(10);
      set.add(20);

      System.out.println(set);
   }
}
```

Output:

[10, 20, 30]

12.7 IMPORTANT MAP CLASSES

1. HASHMAP

HashMap stores elements in key-value pairs.

It does not guarantee insertion order.

Keys must be unique, but values may be duplicated.

2. LINKEDHASHMAP

LinkedHashMap stores key-value pairs and maintains insertion order.

3. TREEMAP

TreeMap stores key-value pairs in sorted order based on keys.

4. HASHTABLE

Hashtable stores key-value pairs and is synchronized.

It does not allow null keys or null values.

12.8 ADVANTAGES OF COLLECTIONS

1. Collections can store multiple objects.

2. They provide dynamic storage.
3. They reduce programming effort.
4. They provide built-in sorting and searching methods.
5. They support different data structures.
6. They improve code reusability.
7. They help manage large amounts of data.

13. INTRODUCTION TO JAVABEANS

13.1 WHAT ARE JAVABEANS?

JavaBeans are reusable software components written in Java.

They are used to create reusable classes that can be easily configured and used in different Java applications.

A JavaBean is a Java class that follows certain standard conventions.

These conventions make the class easier to use with development tools and GUI builders.

Simple Example:

Consider a Student JavaBean.

It contains:

- Student name.
- Student age.
- Methods to set and get the student name.
- Methods to set and get the student age.

The same StudentBean class can be reused in different applications.

13.2 FEATURES OF JAVABEANS

1. Reusable Components: A JavaBean can be reused in different applications.
2. Public Class: A JavaBean class is generally declared as public.
3. No-Argument Constructor: It should provide a public no-argument constructor.
4. Private Properties: Data members are generally declared private.
5. Getter and Setter Methods: Properties are accessed through public getter and setter methods.
6. Serializable: JavaBeans commonly implement the Serializable interface to support persistence.
7. Platform Independent: JavaBeans follow Java's platform-independent nature.
8. Easy Integration: They can be used with suitable GUI builder and development tools.

13.3 ARCHITECTURE OF A JAVABEAN

A JavaBean generally contains three important parts.

1. Properties

Properties represent the data or information stored inside the bean.

Example:

- name
- age
- rollNumber

2. Methods

Methods are used to perform operations and access properties.

Examples:

- setName()
- getName()
- setAge()
- getAge()

3. Events

Events represent actions or changes that can be communicated to other components.

For example, a button click in a GUI application can generate an event.

13.4 IMPORTANT JAVABEAN CONVENTIONS

A JavaBean should follow these conventions:

1. The class should be public.
2. It should have a public no-argument constructor.
3. Properties should generally be private.
4. Getter methods should be used to read property values.
5. Setter methods should be used to modify property values.
6. It should commonly implement Serializable when persistence is required.

GETTER METHOD

A getter method is used to retrieve the value of a private property.

Example:

```
public String getName() {
   return name;
}
```

SETTER METHOD

A setter method is used to assign a value to a private property.

Example:

```
public void setName(String name) {
   this.name = name;
}
```

13.5 JAVA PROGRAM: CREATING A SIMPLE JAVABEAN

```
import java.io.Serializable;

public class StudentBean implements Serializable {

    private String name;
    private int age;

    // No-argument constructor
    public StudentBean() {
    }

    // Getter method
    public String getName() {
        return name;
    }

    // Setter method
    public void setName(String name) {
        this.name = name;
    }

    // Getter method
    public int getAge() {
        return age;
    }

    // Setter method
    public void setAge(int age) {
        this.age = age;
    }

    // Display method
    public String display() {
        return "Name: " + name + ", Age: " + age;
```

```
    }
}
```

MAIN CLASS

```
public class TestBean {

    public static void main(String[] args) {

        StudentBean bean = new StudentBean();

        bean.setName("Rahul");
        bean.setAge(20);

        System.out.println(bean.display());
    }
}
```

Output:

Name: Rahul, Age: 20

EXPLANATION

1. StudentBean is declared as a public class.
2. The Serializable interface is implemented.
3. The name and age properties are declared private.
4. A public no-argument constructor is created.
5. Getter methods retrieve the values.
6. Setter methods assign values.
7. The main class creates the JavaBean object.
8. Values are assigned using setter methods.
9. The details are displayed.

13.6 WORKING OF JAVABEANS

The working process of a JavaBean is as follows:

1. A JavaBean class is created.
2. An object of the bean is created using the no-argument constructor.
3. Values are assigned using setter methods.
4. Values are retrieved using getter methods.
5. The bean is used in a Java application or supported development tool.

13.7 ADVANTAGES OF JAVABEANS

1. Supports reusable software components.
2. Reduces code duplication.
3. Makes application development easier.
4. Provides encapsulation through private properties.

5. Supports serialization.
6. Makes components easier to configure and maintain.
7. Can be integrated with suitable GUI development tools.

Applications of JavaBeans:

- Student management systems.
- Employee management applications.
- Desktop GUI applications.
- Reusable application components.
- Java-based enterprise applications.

14. NETWORK PROGRAMMING IN JAVA

14.1 WHAT IS NETWORK PROGRAMMING?

Network programming is the process of developing applications that communicate with each other through a computer network.

Java provides networking classes and interfaces through the java.net package.

Network programming allows two or more computers to exchange data using communication protocols.

Simple Example:

When we use a chat application:

1. One user sends a message.
2. The message travels through a network.
3. The receiving computer receives the message.
4. The message is displayed to the receiver.

This type of communication is possible through network programming.

APPLICATIONS OF NETWORK PROGRAMMING

1. Chat applications.
2. Online banking systems.
3. Web applications.
4. File-sharing applications.
5. Multiplayer online games.
6. Client-server applications.
7. Email applications.

14.2 IMPORTANT CLASSES IN THE JAVA.NET PACKAGE

Java provides several classes for network communication.

CLASS	DESCRIPTION
Socket	Establishes a connection between a client and server.

ServerSocket	Creates a server that listens for client connections.
InetAddress	Represents an IP address.
URL	Represents a Uniform Resource Locator.
URLConnection	Represents a communication link to a URL resource.
DatagramSocket	Sends and receives UDP packets.
DatagramPacket	Represents a UDP packet of data.

1. SOCKET CLASS

The Socket class is used to establish a communication connection between a client and a server.

It is mainly used in TCP-based communication.

2. SERVERSOCKET CLASS

The ServerSocket class is used on the server side.

It listens for incoming client connection requests.

When a client connects, the server accepts the connection and creates a Socket object for communication.

3. INETADDRESS CLASS

InetAddress represents an Internet Protocol (IP) address.

It can be used to obtain information about a host name and its IP address.

4. URL CLASS

URL stands for Uniform Resource Locator.

It represents the address of a resource available on the internet or another network.

Example:

https://www.example.com

5. URLCONNECTION CLASS

URLConnection is used to establish communication with a resource identified by a URL.

It can be used to read data from web resources.

6. DATAGRAMSOCKET CLASS

DatagramSocket is used for sending and receiving UDP packets.

UDP communication is connectionless.

7. DATAGRAMPACKET CLASS

DatagramPacket represents a packet of data sent or received using UDP.

14.3 TYPES OF NETWORK COMMUNICATION

Java mainly supports two important network communication protocols.

1. TCP (TRANSMISSION CONTROL PROTOCOL)

TCP is a connection-oriented communication protocol.

Before exchanging data, a connection is established between the client and server.

Features:

1. Connection-oriented.
2. Reliable communication.
3. Maintains data order.
4. Provides error checking and retransmission.
5. Suitable for applications where accurate data delivery is important.

Examples:

- Web browsing using HTTP/HTTPS over TCP.
- File transfer.
- Email communication.

2. UDP (USER DATAGRAM PROTOCOL)

UDP is a connectionless communication protocol.

It sends data in the form of packets called datagrams without establishing a connection before every communication.

Features:

1. Connectionless communication.
2. Faster with lower communication overhead.
3. Does not guarantee delivery.
4. Does not guarantee packet order.
5. Suitable for real-time communication.

Examples:

- Online gaming.
- Live video streaming.

- Voice communication.

14.4 DIFFERENCE BETWEEN TCP AND UDP

TCP	UDP
Connection-oriented.	Connectionless.
Reliable data delivery.	Delivery is not guaranteed.
Maintains data order.	Does not guarantee data order.
Generally slower due to additional control mechanisms.	Generally faster with lower overhead.
Uses Socket and ServerSocket classes.	Uses DatagramSocket and DatagramPacket classes.
Suitable for file transfer and web communication.	Suitable for real-time applications.

14.5 CLIENT-SERVER MODEL

The client-server model is a network architecture in which one computer requests services and another computer provides those services.

There are two main components.

1. Client: A client is a program or computer that requests data or services from a server.

Examples:

- Web browser.
- Mobile application.
- Desktop application.

2. Server: A server is a program or computer that receives client requests, processes them and sends responses.

Examples:

- Web server.
- Database server.
- File server.

WORKING OF THE CLIENT-SERVER MODEL

1. The server starts and waits for client connections.
2. The client creates a connection request.
3. The server accepts the client connection.
4. The client sends data or a request.
5. The server receives and processes the request.
6. The server sends a response.
7. The client receives and displays the response.
8. The connection is closed when communication is completed.

14.6 SOCKET COMMUNICATION IN JAVA

Socket communication is a method of exchanging information between two computers using network connections.

Java supports socket communication using the Socket and ServerSocket classes.

There are two sides:

Client Side

- Creates a Socket object.
- Connects to the server.
- Sends data.
- Receives data.

Server Side

- Creates a ServerSocket object.
- Waits for a client.
- Accepts the connection.
- Receives data.
- Sends a response.

14.7 JAVA PROGRAM: SIMPLE CLIENT-SERVER APPLICATION

For this example, we create two Java programs:

1. Server Program.
2. Client Program.

Both programs communicate using TCP socket communication.

A. SERVER PROGRAM

```
import java.io.*;

import java.net.*;


public class ChatServer {


   public static void main(String[] args) {


      try {


         ServerSocket server =

            new ServerSocket(5000);


         System.out.println("Server is waiting for client...");


         Socket socket = server.accept();


         System.out.println("Client connected.");
```

```
            BufferedReader in =
               new BufferedReader(
                  new InputStreamReader(
                     socket.getInputStream()));

            PrintWriter out =
               new PrintWriter(
                  socket.getOutputStream(), true);

            String message = in.readLine();

            System.out.println("Client: " + message);

            out.println("Hello from Server!");

            socket.close();
            server.close();

         }
         catch(IOException e) {
            System.out.println("Error: " + e.getMessage());
         }
      }
   }
```

B. CLIENT PROGRAM

```
import java.io.*;
import java.net.*;

public class ChatClient {

   public static void main(String[] args) {

      try {

         Socket socket =
```

```
            new Socket("localhost", 5000);

        System.out.println("Connected to server.");

        PrintWriter out =
            new PrintWriter(
                socket.getOutputStream(), true);

        BufferedReader in =
            new BufferedReader(
                new InputStreamReader(
                    socket.getInputStream()));

        out.println("Hello from Client!");

        String response = in.readLine();

        System.out.println("Server: " + response);

        socket.close();

      }
      catch(IOException e) {
        System.out.println("Error: " + e.getMessage());
      }
    }
}
```

OUTPUT

Server Output

Server is waiting for client...

Client connected.

Client: Hello from Client!

Client Output

Connected to server.

Server: Hello from Server!

EXPLANATION OF THE SERVER PROGRAM

1. The java.net package is imported for networking classes.
2. ServerSocket is created on port number 5000.
3. The server waits for a client using accept().
4. When a client connects, the server creates a Socket object.
5. BufferedReader receives the client's message.
6. PrintWriter sends the response to the client.
7. The server closes the socket after communication.

EXPLANATION OF THE CLIENT PROGRAM

1. A Socket object is created.
2. The client connects to the server running on localhost.
3. PrintWriter is used to send a message.
4. BufferedReader is used to receive the server response.
5. The received message is displayed.
6. The socket is closed.

Important: Run the server program first and then run the client program. Both programs should be running on the same computer for this localhost example.

14.8 ADVANTAGES OF NETWORK PROGRAMMING

1. Allows communication between computers.
2. Supports data sharing over networks.
3. Helps develop client-server applications.
4. Supports TCP and UDP protocols.
5. Useful for internet-based applications.
6. Supports real-time communication.
7. Helps develop distributed applications.